

✓ 🎓 LISTA DE EXERCÍCIOS 1: PyTorch Tensores

Parte 1 - Fundamentos

Dificuldade: Básica a Intermediária **Tempo estimado:** 60-90 minutos **Exercícios:** 16 (Níveis 1-4)

Objetivos

- ✓ Criar tensores de várias formas
- ✓ Entender tipos de dados
- ✓ Manipular shapes (reshape, flatten, etc)
- ✓ Realizar operações algébricas básicas

Próximo

Após completar esta lista, passe para **LISTA DE EXERCÍCIOS 2** para exercícios avançados!

✓ Setup

```
import torch
import numpy as np

print(f'PyTorch versão: {torch.__version__}')
```

PyTorch versão: 2.11.0+cpu

✓ NÍVEL 1: Criação Básica de Tensores

Aprenda as formas fundamentais de criar tensores

✓ 📄 EXERCÍCIO 1.1: Tensor vazio

Tarefa:

- Crie um tensor vazio com shape (3, 4)
- Imprima o tensor
- Imprima seu shape
- Imprima seu dtype

```
t = torch.empty(3, 4)

print(t)
print("shape:", t.shape)
print("dtype:", t.dtype)

tensor([[ 7.8402e-31,  0.0000e+00,  7.8380e-31,  0.0000e+00],
        [ 1.1210e-43,  0.0000e+00,  8.9683e-44,  0.0000e+00],
        [ 3.8896e-31,  0.0000e+00,  0.0000e+00,  1.5046e-36]])
shape: torch.Size([3, 4])
dtype: torch.float32
```

✓ 📄 EXERCÍCIO 1.2: Zeros e Ones

Tarefa:

1. Crie um tensor de zeros com shape (2, 3)
2. Crie um tensor de ones com shape (2, 3)
3. Imprima ambos
4. Some os dois tensores

```
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)

print("zeros:\n", zeros)
print("ones:\n", ones)

print("soma:\n", zeros + ones)
```

```
zeros:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
ones:
tensor([[1., 1., 1.],
        [1., 1., 1.]])
soma:
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

EXERCÍCIO 1.3: Tensor com valor constante

Tarefa:

1. Crie um tensor com shape (4, 4) preenchido com 5.0
2. Crie um tensor com shape (3, 3) preenchido com 7
3. Imprima o dtype de cada um
4. Qual é a diferença?

```
a = torch.full((4, 4), 5.0)
b = torch.full((3, 3), 7)

print("a.dtype:", a.dtype)
print("b.dtype:", b.dtype)
```

```
a.dtype: torch.float32
b.dtype: torch.int64
```

EXERCÍCIO 1.4: Lista para tensor

Tarefa:

1. Crie lista: [1, 2, 3, 4, 5]
2. Converta para tensor
3. Imprima o tensor e seu dtype
4. Crie novamente com dtype=torch.float32
5. Compare os dois

```
lista = [1, 2, 3, 4, 5]

t_int = torch.tensor(lista)
print(t_int, "| dtype:", t_int.dtype)

t_float = torch.tensor(lista, dtype=torch.float32)
print(t_float, "| dtype:", t_float.dtype)
```

```
tensor([1, 2, 3, 4, 5]) | dtype: torch.int64
tensor([1., 2., 3., 4., 5.]) | dtype: torch.float32
```

NÍVEL 2: Tipos de Tensores (dtype)

EXERCÍCIO 2.1: Diferentes dtypes

Tarefa: Crie um tensor com valor 42 em cada tipo:

- torch.int32
- torch.int64

- torch.float32
- torch.float64
- torch.bool

Para cada um, imprima:

- O tensor
- Seu dtype
- Seu tamanho em bytes

```
tipos = [torch.int32, torch.int64, torch.float32, torch.float64, torch.bool]

for dt in tipos:
    t = torch.tensor(42, dtype=dt)
    nbytes = t.element_size() * t.numel()
    print(f"{str(dt):15} -> valor={t.item():s:6} | bytes={nbytes}")
```

```
torch.int32    -> valor=42      | bytes=4
torch.int64    -> valor=42      | bytes=8
torch.float32  -> valor=42.0    | bytes=4
torch.float64  -> valor=42.0    | bytes=8
torch.bool     -> valor=True    | bytes=1
```

EXERCÍCIO 2.2: Conversão entre tipos

Tarefa:

1. Crie: `t1 = torch.tensor([1.5, 2.5, 3.5])`
2. Converta para int32: `t2 = t1.to(torch.int32)`
3. Imprima ambos
4. O que aconteceu?
5. Converta de volta para float64

```
t1 = torch.tensor([1.5, 2.5, 3.5])
t2 = t1.to(torch.int32)

print("t1:", t1)
print("t2:", t2)

t3 = t2.to(torch.float64)
print("t3:", t3, "| dtype:", t3.dtype) # informação já foi perdida no truncamento

t1: tensor([1.5000, 2.5000, 3.5000])
t2: tensor([1, 2, 3], dtype=torch.int32)
t3: tensor([1., 2., 3.], dtype=torch.float64) | dtype: torch.float64
```

EXERCÍCIO 2.3: Verificar tipo

Tarefa:

1. Crie 3 tensores com tipos diferentes
2. Para cada um, imprima:
 - `t.dtype`
 - `t.dtype == torch.float32`
3. Crie função que retorna "int" ou "float" ou "outro"

```
ta = torch.tensor([1, 2, 3])
tb = torch.tensor([1.0, 2.0])
tc = torch.tensor([True, False])

for t in (ta, tb, tc):
    print(t.dtype, "| é float32?", t.dtype == torch.float32)

def categoria(t: torch.Tensor) -> str:
    if t.dtype.is_floating_point:
        return "float"
    if t.dtype in (torch.int8, torch.int16, torch.int32, torch.int64, torch.uint8):
        return "int"
```

```

        return "outro"

print(categoria(ta), categoria(tb), categoria(tc)) # int float outro

torch.int64 | é float32? False
torch.float32 | é float32? True
torch.bool | é float32? False
int float outro

```

✓ NÍVEL 3: Manipulação de Shape

✓ EXERCÍCIO 3.1: Shape e dimensões

Tarefa:

1. Crie tensor com shape (2, 3, 4)
2. Imprima:
 - tensor.shape
 - tensor.size()
 - len(tensor.shape)
 - tensor.numel() (total elementos)
 - tensor.dim() (dimensões)

```

t = torch.zeros(2, 3, 4)

print("shape: ", t.shape)
print("size(): ", t.size())
print("len(shape): ", len(t.shape))
print("numel(): ", t.numel())
print("dim(): ", t.dim())

shape: torch.Size([2, 3, 4])
size(): torch.Size([2, 3, 4])
len(shape): 3
numel(): 24
dim(): 3

```

✓ EXERCÍCIO 3.2: Reshape

Tarefa:

1. Crie: torch.arange(12)
2. Reshape para (3, 4)
3. Reshape para (2, 2, 3)
4. Reshape para (1, 12)
5. Imprima o shape em cada etapa

```

t = torch.arange(12)
print("original:", t.shape)

a = t.reshape(3, 4)
print("(3,4): ", a.shape)

b = t.reshape(2, 2, 3)
print("(2,2,3): ", b.shape)

c = t.reshape(1, 12)
print("(1,12): ", c.shape)

original: torch.Size([12])
(3,4): torch.Size([3, 4])
(2,2,3): torch.Size([2, 2, 3])
(1,12): torch.Size([1, 12])

```

✓ EXERCÍCIO 3.3: Flatten e Unflatten

Tarefa:

1. Crie tensor com shape (3, 4, 2)
2. Use flatten() e view(-1)
3. Qual é a diferença?
4. Converta de volta para (3, 4, 2)

```
t = torch.arange(24).reshape(3, 4, 2)

f1 = t.flatten()
f2 = t.view(-1)

print("flatten:", f1.shape, "| view(-1):", f2.shape)

back = f1.reshape(3, 4, 2)
print("de volta:", back.shape)

flatten: torch.Size([24]) | view(-1): torch.Size([24])
de volta: torch.Size([3, 4, 2])
```

✓ EXERCÍCIO 3.4: Squeeze e Unsqueeze**Tarefa:**

1. Crie tensor com shape (3, 1, 4)
2. Use squeeze()
3. Use unsqueeze(0) e unsqueeze(-1)
4. Imprima shape em cada etapa

```
t = torch.zeros(3, 1, 4)
print("original: ", t.shape)

s = t.squeeze()
print("squeeze(): ", s.shape)

u0 = s.unsqueeze(0)
print("unsqueeze(0): ", u0.shape)

um1 = s.unsqueeze(-1)
print("unsqueeze(-1): ", um1.shape)

original: torch.Size([3, 1, 4])
squeeze(): torch.Size([3, 4])
unsqueeze(0): torch.Size([1, 3, 4])
unsqueeze(-1): torch.Size([3, 4, 1])
```

✓ NÍVEL 4: Operações Algébricas

✓ EXERCÍCIO 4.1: Operações elemento a elemento**Tarefa:**

1. Crie:
 - `t1 = torch.tensor([1, 2, 3, 4])`
 - `t2 = torch.tensor([5, 6, 7, 8])`
2. Calcule:
 - `t1 + t2`
 - `t1 - t2`
 - `t1 * t2`
 - `t1 / t2`
 - `t1 ** 2`

```
t1 = torch.tensor([1, 2, 3, 4])
t2 = torch.tensor([5, 6, 7, 8])

print("t1 + t2 :", t1 + t2)
print("t1 - t2 :", t1 - t2)
print("t1 * t2 :", t1 * t2)
print("t1 / t2 :", t1 / t2)
print("t1 ** 2 :", t1 ** 2)
```

```
t1 + t2 : tensor([ 6,  8, 10, 12])
t1 - t2 : tensor([-4, -4, -4, -4])
t1 * t2 : tensor([ 5, 12, 21, 32])
t1 / t2 : tensor([0.2000, 0.3333, 0.4286, 0.5000])
t1 ** 2 : tensor([ 1,  4,  9, 16])
```

EXERCÍCIO 4.2: Operações com escalar

Tarefa:

1. Crie: `t = torch.tensor([1.0, 2.0, 3.0, 4.0])`
2. Calcule:
 - `t + 10`
 - `t * 2`
 - `t / 2`
 - `100 / t`
3. O tensor original mudou?

```
t = torch.tensor([1.0, 2.0, 3.0, 4.0])

print("t + 10 :", t + 10)
print("t * 2 :", t * 2)
print("t / 2 :", t / 2)
print("100 / t:", 100 / t)

print("original:", t)
```

```
t + 10 : tensor([11., 12., 13., 14.])
t * 2 : tensor([2., 4., 6., 8.])
t / 2 : tensor([0.5000, 1.0000, 1.5000, 2.0000])
100 / t: tensor([100.0000,  50.0000,  33.3333,  25.0000])
original: tensor([1., 2., 3., 4.])
```

EXERCÍCIO 4.3: Operações In-Place

Tarefa:

1. Crie: `t = torch.tensor([1.0, 2.0, 3.0])`
2. Imprima o tensor
3. Use `t.add_(10)`
4. Imprima novamente - mudou?
5. Teste também: `t.mul_(2)`, `t.sub_(1)`

```
t = torch.tensor([1.0, 2.0, 3.0])
print("antes: ", t)

t.add_(10)
print("add_(10):", t)

t.mul_(2)
print("mul_(2): ", t)

t.sub_(1)
print("sub_(1): ", t)
```

```
antes: tensor([1., 2., 3.])
add_(10): tensor([11., 12., 13.])
mul_(2): tensor([22., 24., 26.])
sub_(1): tensor([21., 23., 25.])
```

EXERCÍCIO 4.4: Redução

Tarefa:

1. Crie: `t = torch.tensor([1, 2, 3, 4, 5])`
2. Calcule e imprima:
 - `t.sum()`
 - `t.mean()`
 - `t.max()`
 - `t.min()`
3. Use `.item()` para extrair valor Python

```
t = torch.tensor([1, 2, 3, 4, 5])

print("sum:", t.sum())
print("max:", t.max())
print("min:", t.min())

print("mean:", t.float().mean())

print("sum como float Python:", t.sum().item(), type(t.sum().item()))
```

```
sum: tensor(15)
max: tensor(5)
min: tensor(1)
mean: tensor(3.)
sum como float Python: 15 <class 'int'>
```

EXERCÍCIO 4.5: Produto de Matrizes

Tarefa:

1. Crie:
 - `A = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)`
 - `B = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)`
2. Calcule: `A @ B`
3. Qual é o shape?
4. Verifique manualmente

```
A = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
B = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)

C = A @ B
print(C)
print("shape:", C.shape)

# Verificação manual:
# C[0,0] = 1*5 + 2*7 = 19 ; C[0,1] = 1*6 + 2*8 = 22
# C[1,0] = 3*5 + 4*7 = 43 ; C[1,1] = 3*6 + 4*8 = 50
assert torch.equal(C, torch.tensor([[19., 22.], [43., 50.])))
print("✓ conferido manualmente")
```

```
tensor([[19., 22.],
        [43., 50.]])
shape: torch.Size([2, 2])
conferido manualmente
```

